



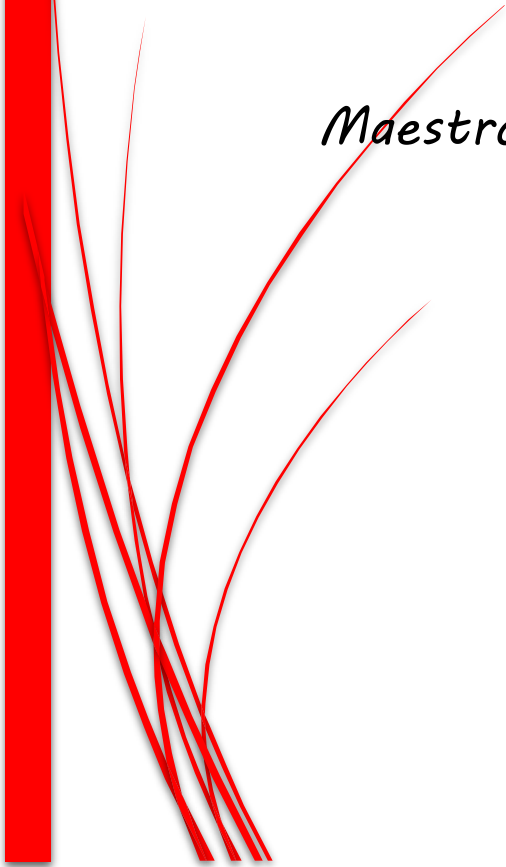
Universidad Tecnológica De Nezahualcóyotl

Documentación Base de Datos y Streamer

Alumno(a): Solis Garcia Fany Sarai

Materia: Aplicaciones Web Orientada a Objetos

Maestro: Del Prado López Jovan



1 Estructura General del Proyecto

APIs utilizadas en VideoTube

YouTube Embed API Cuando el usuario pega un link de YouTube en el formulario de agrega.html, el archivo pasatiempo-agrega.php detecta si la URL contiene watch?v= oyoutu.be/, extrae el ID del video y construye la URL en formato embed:

`https://www.youtube.com/embed/ID_DEL_VIDEO`

Esa URL se guarda en la base de datos y luego se usa como fuente de un iframe para reproducir el video dentro de la página. No requiere clave ni registro, es la versión pública de embed de YouTube.

¿Qué es VideoTube?

VideoTube es una **plataforma de streaming personal** desarrollada en PHP y MySQL. Permite subir videos propios o enlazar videos de YouTube, organizarlos por categoría (Miedo, Novela, Comedia, Música, Podcast, Comida), gestionar usuarios registrados y navegar el catálogo desde una interfaz inspirada en YouTube.

Mapa de archivos del proyecto

Archivo	Tipo	Función principal
conexion.php	PHP BD	Establece la conexión a la base de datos MySQL en InfinityFree
pasatiempo-agrega.php	PHP BD	Recibe el formulario de subida y guarda el video en la BD
pasatiempo-edita.php	PHP BD	Actualiza los datos de un video existente en la BD
elimina.php	PHP BD	Elimina un video de la BD y su archivo físico si existe
eliminar_usuario.php	PHP BD	Elimina un usuario de la tabla USUARIOS
actualizar_usuario.php	PHP BD	Actualiza el nombre de un usuario en la BD
index.php	PHP Vista	Página principal: muestra videos, usuarios y filtros por categoría
agrega.html	HTML Vista	Formulario para subir un video (archivo MP4 o link de YouTube)
edita.php	PHP Vista	Formulario prellenado para editar un video existente
gestionar_usuario.php	PHP Vista	Panel para modificar o eliminar un usuario

registro.php	PHP Vista	/ Formulario de registro de nuevos usuarios

2.1 conexion.php

Es el archivo más importante del backend. Establece la conexión con la base de datos MySQL alojada en InfinityFree usando la clase mysqli. Si la conexión falla, detiene todo con die(). Además configura el charset utf8mb4 para que los acentos y caracteres especiales se muestren correctamente.

```
$con = new mysqli($host, $user, $pass, $db); if ($con->connect_error) { die('Error: ' . $con->connect_error); } $con->set_charset('utf8mb4');
```

⚠ Este archivo se incluye con include('conexion.php') en todos los demás archivos PHP. Sin él ninguna operación de base de datos funciona.

2.2 pasatiempo-agrega.php

Recibe los datos del formulario agrega.html y guarda el video en la tabla PASATIEMPO. Tiene dos modos: si el usuario pega un link de YouTube, el código lo convierte automáticamente al formato embed (watch?v= se reemplaza por embed/). Si en cambio sube un archivo MP4, lo mueve a la carpeta videos/ con un nombre único basado en time().

```
// Convierte URL normal a embed if (strpos($link_externo, 'watch?v=') !== false) { $id = explode('v=', $link_externo)[1]; $ruta_final = 'https://www.youtube.com/embed/' . $id; } // Guarda en BD $stmt = $con->prepare('INSERT INTO PASATIEMPO (titulo, ruta_video, categoria) VALUES (?, ?, ?)'); $stmt->bind_param('sss', $titulo, $ruta_final, $categoria); $stmt->execute();
```

⚠ bind_param('sss...') usa sentencias preparadas para proteger contra inyección SQL. La 's' indica que el parámetro es de tipo string.

2.3 pasatiempo-edita.php

Recibe el formulario de edición y actualiza los datos del video en la BD usando UPDATE. También convierte links de YouTube al formato embed si el usuario pega una URL normal.

```
if (strpos($ruta_video, 'watch?v=') !== false) { $ruta_video = str_replace('watch?v=', 'embed/', $ruta_video); } $stmt = $con->prepare('UPDATE PASATIEMPO SET titulo=?, ruta_video=?, categoria=? WHERE id=?'); $stmt->bind_param('sssi', $titulo, $ruta_video, $categoria, $id); $stmt->execute();
```

⚠ 'sssi' indica 3 strings y 1 entero (integer). El tipo de cada parámetro debe coincidir exactamente con el orden en bind_param.

2.4 elimina.php

Elimina un video de forma segura en dos pasos: primero busca la ruta del video en la BD para saber si es un archivo local. Si es un archivo MP4 local (no YouTube) y existe en el servidor, lo borra con `unlink()`. Después elimina el registro de la BD con `DELETE`.

```
// Paso 1: buscar y borrar archivo físico $stmt_select = $con->prepare('SELECT ruta_video
FROM PASATIEMPO WHERE id=?'); $stmt_select->bind_param('i', $id); $stmt_select-
>execute(); $fila = $stmt_select->get_result()->fetch_assoc(); if (file_exists($ruta) &&
strpos($ruta, 'youtube.com') === false) { unlink($ruta); // Borra el archivo del servidor } // Paso
2: borrar de la BD $stmt_delete = $con->prepare('DELETE FROM PASATIEMPO WHERE id=?');
$stmt_delete->bind_param('i', $id); $stmt_delete->execute();
```

⚠ Si el video es de YouTube solo se elimina el registro, no hay archivo físico que borrar. `file_exists()` y `strpos()` combinados evitan borrar links por error.

2.5 eliminar_usuario.php

Recibe el ID del usuario por GET y lo elimina de la tabla `USUARIOS` con una consulta `DELETE` directa. Tras eliminar, redirige al `index`.

```
$id = $_GET['id']; if ($con->query('DELETE FROM USUARIOS WHERE id = $id')) {
header('Location: ../index.php'); }
```

⚠ A diferencia de los otros archivos, este usa una consulta directa (sin `prepare`). Para mayor seguridad se recomienda usar sentencias preparadas también aquí.

2.6 actualizar_usuario.php

Recibe el ID y el nuevo nombre del usuario por POST y ejecuta un `UPDATE` sobre la tabla `USUARIOS`. Al terminar muestra una alerta JavaScript y redirige al `index`.

```
$sql = "UPDATE USUARIOS SET nombre = '$nombre' WHERE id = $id"; if ($con->query($sql)) {
echo "<script>alert('Datos actualizados'); window.location='../index.php';</script>"; }
```

⚠ La alerta se genera con `echo` de código JavaScript, una técnica sencilla para mostrar confirmación sin recargar la página con una nueva vista.

3.1 index.php — Página principal

Es la página de inicio de VideoTube. Tiene tres secciones: la barra de navegación superior con el logo y el botón de subir video, una barra lateral izquierda que muestra la lista de usuarios registrados (o un cuadro de bienvenida si ya se seleccionó uno), y el contenido principal con los filtros por categoría y la cuadrícula de videos cargados desde la BD.

```
// Filtro de categoría por GET $cat_filtro = isset($_GET['categoria']) ? $_GET['categoria'] : ""; //
Sesión de usuario (sin PHP sessions, por URL) $user_id = isset($_GET['user_id']) ?
$_GET['user_id'] : ""; if ($user_id != "") { $res_u = $con->query("SELECT nombre FROM
USUARIOS WHERE id = '$user_id'"); $nombre_usuario_activo = $res_u-
>fetch_assoc()['nombre']; }
```

Bloque CSS	Propósito
nav	Barra superior pegajosa (sticky) con logo y botón de subir
.sidebar	Columna izquierda con lista de usuarios o cuadro de bienvenida
.welcome-box	Aparece cuando hay un user_id activo en la URL
.categories	Filtros de categoría en forma de pastillas clickeables
.video-grid	Cuadrícula CSS auto-fill de tarjetas de video

⚠ La 'sesión' no usa PHP \$_SESSION sino parámetros en la URL (user_id=). Es una solución sencilla que se preserva al navegar entre páginas.

3.2 agrega.html — Subir video

Formulario para publicar un video nuevo. Tiene dos opciones: subir un archivo MP4 desde el dispositivo (máximo 2MB) o pegar un link de YouTube. El formulario usa enctype='multipart/form-data' para poder enviar archivos, y señala al script pasatiempo-agrega.php como destino.

```
<form action='php/pasatiempo-agrega.php' method='POST' enctype='multipart/form-data'>
<!-- Opción A: archivo MP4 --> <input type='file' name='video' accept='video/mp4'> <!--
Opción B: link de YouTube --> <input type='text' name='link_externo'
placeholder='https://www.youtube.com/watch?v=...> </form>
```

Campo formulario	del	Nombre en PHP	Función
Título del video		\$_POST['titulo']	Nombre con el que se guarda en la BD
Categoría		\$_POST['categoria']	Define el filtro al que pertenece el video

Archivo MP4	\$_FILES['video']	Archivo físico que se mueve a la carpeta videos/
Link de YouTube	\$_POST['link_externo']	URL que el PHP convierte a formato embed

⚠ `enctype='multipart/form-data'` es obligatorio para subir archivos. Sin ese atributo, `$_FILES` siempre estará vacío.

3.3 edita.php — Editar video

Carga los datos actuales del video desde la BD y los muestra prellenados en un formulario. El usuario puede cambiar el título, la ruta o link y la categoría. Al guardar, envía los datos a `pasatiempo-edita.php`.

```
// Recupera el video de la BD $stmt = $con->prepare('SELECT * FROM PASATIEMPO WHERE
id=?'); $stmt->bind_param('i', $id); $stmt->execute(); $video = $stmt->get_result()-
>fetch_assoc(); // Prellenado en el HTML <input name='titulo' value='<?=
htmlspecialchars($video['titulo']) ?>'>
```

⚠ `htmlspecialchars()` convierte caracteres como `<` `>` `&` en entidades HTML seguras, evitando que el contenido de la BD rompa el formulario.

3.4 gestionar_usuario.php — Modificar usuario

Panel de administración de un usuario. Muestra su nombre en un campo editable y tiene dos botones: Guardar (envía a `actualizar_usuario.php`) y Eliminar (enlaza a `eliminar_usuario.php` con confirmación JavaScript).

```
// Botones de acción <button type='submit' class='btn-save'>Guardar</button> <a
href='php/eliminar_usuario.php?id=<?= $user['id'] ?>' onclick='return confirm("&Seguro que
quieres eliminar?")' class='btn-delete'>Eliminar</a>
```

⚠ El `confirm()` de JavaScript muestra un diálogo nativo del navegador antes de proceder. Si el usuario cancela, el enlace no se ejecuta.

3.5 registro.php — Crear cuenta

Formulario de registro de nuevos usuarios. Solicita nombre, correo electrónico y contraseña. Los datos se envían por POST al script `procesar-registro.php`. Tiene validación HTML5 (`required`) y efectos visuales al enfocar los campos.

```
<form action='php/procesar-registro.php' method='POST'> <input type='text'
name='nombre' required> <input type='email' name='correo' required> <input
```

```
type='password' name='password' required>    <button type='submit'>Crear cuenta</button>
</form>
```

Campo	Tipo HTML	Validación
Nombre	text	Campo requerido, no puede estar vacío
Correo	email	El navegador valida que tenga formato de email (contiene @)
Contraseña	password	Requerida; el texto se oculta automáticamente

⚠ type='email' y type='password' son tipos nativos de HTML5. El navegador aplica validación básica sin necesidad de JavaScript.

4 Flujo Completo de la Aplicación

Flujo para subir un video

#	Actor	Acción
1	Usuario	Abre agrega.html y llena el formulario con título, categoría y video/link
2	HTML	Valida los campos con required y envía el form a pasatiempo-agrega.php por POST
3	pasatiempo-agrega.php	Detecta si hay link de YouTube o archivo MP4
4	PHP (YouTube)	Extrae el ID del video y construye la URL embed de YouTube
4	PHP (MP4)	Genera un nombre único con time() y mueve el archivo a la carpeta videos/
5	PHP	Guarda título, ruta_final y categoría en la tabla PASATIEMPO con INSERT
6	PHP	Redirige a index.php donde el video ya aparece en la cuadrícula

Flujo para eliminar un video

#	Actor	Acción
1	Usuario	Hace clic en el botón Eliminar de una tarjeta de video
2	HTML	Envía el id del video por GET a elimina.php
3	elimina.php	Consulta la ruta del video en la BD con SELECT
4	PHP	Si la ruta es un archivo local (no YouTube), borra el archivo con unlink()
5	PHP	Ejecuta DELETE en la tabla PASATIEMPO
6	PHP	Redirige a index.php; el video ya no aparece

⚠ Los flujos de editar usuario y registrar usuario siguen el mismo patrón: formulario HTML → script PHP → consulta BD → redirección a index.php.